Servlets & JSPs

Background of Servlet: the Applet

- An optimal *distributed approach* led to the proposal of executing code on the browsers instead of the servers.
- It required a tool to *dynamically and incrementally* download the application to be executed on the browser, besides, the "sand box" concept had to be satisfied in order to overcome any security risk.
- It led Sun Microsystems to make a proposal using an *object oriented programming language*, like Java, and its java virtual machine as the "*sand box*" approach to be included within any browser, and those downloaded applications were named as "Applets".
- Nevertheless, a non extensive use of this technology led to think it over, and finally it was decided to move the code execution to the servers, talking about "Servlets" instead of "Applets".

The Servlet

- The Servlets are execute on web servers, called **servlet containers**, and more recently, application servers, indeed application containers, as far as its functionality goes beyond the Servlets execution, e.g. as a message broker.
- They replace the original CGIs to improve their execution and make application developing more easy.
- In any case, web servers are traditionally considered as ***static*** content providers, while the application servers are ***dynamic*** content providers.

Servlet aim

- Basically we require a tool to implement a client/server HTTP protocol dialogue:
  - a tool to obtain any information contained in an HTTP **request**.
  - a tool to compose the HTTP **reply** to send back to the client.

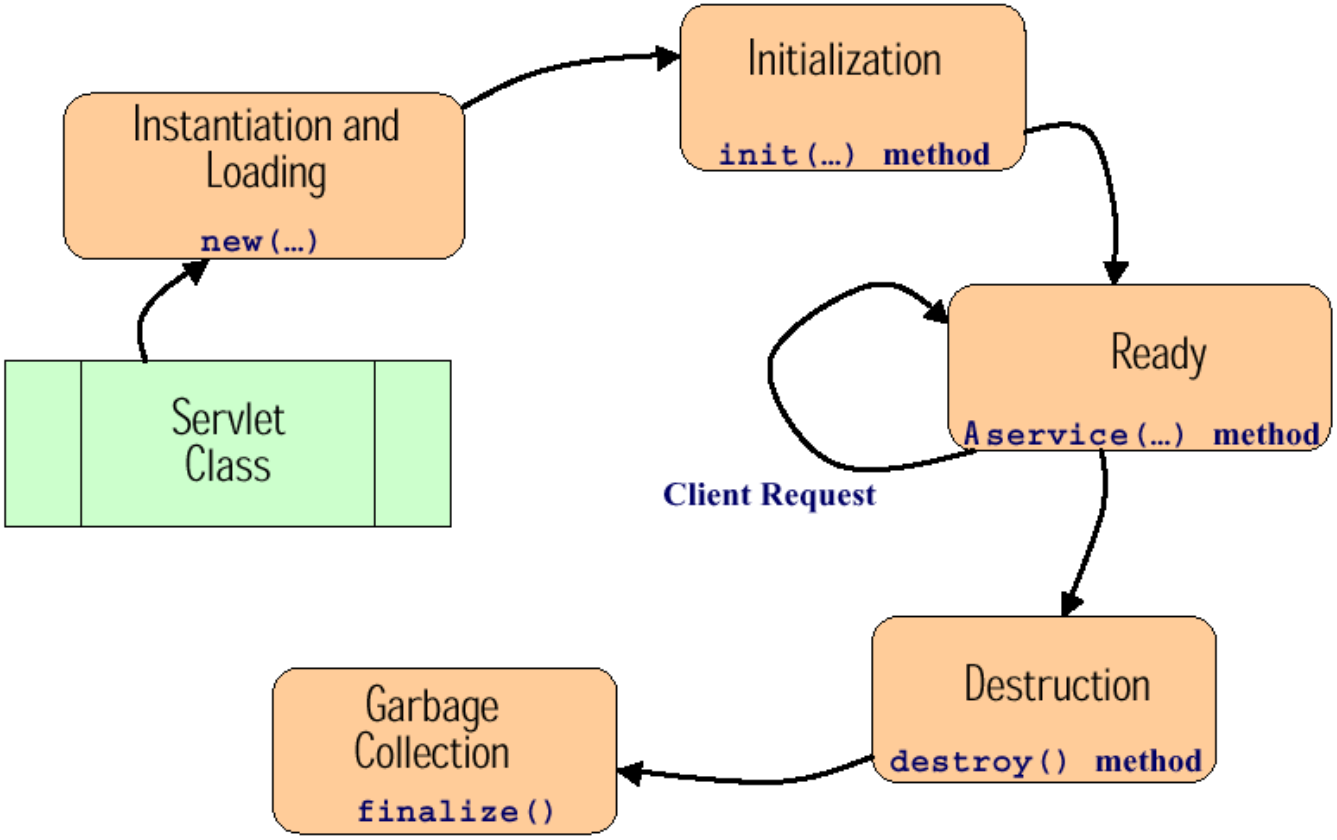
Servlet's requested features

- To handle objects with ***different scopes*** to implement any kind of web application
- A mechanism to handle ***multiple requests*** made in the ***same session*** by the same client
- Support for multiple ***simultaneous requests*** from ***different clients*** to the same Servlet. So, any concurrency inconsistency must be solved.
- It would be desirable to have the option to ***distribute the processing*** of the request among different Servlets, ***forwarding the request*** among them.

Servlets – cycle of life

- Any Servlet has a cycle of life including:
  - The Servlet **load**
  - The **processing** of requests
  - The Servlet **unload**
- The *Servlet container* handles the Servlet cycle of life. We do not need to know the details as Servlet programmers.

Servlets – Life cycle



Servlets – Life cycle

- The Servlets are loaded/unloaded by the Servlet container. The container decides when to do that, on container start up, on servlet's first request, etc.
- On loading any Servlet, the init() method is executed, but the init() method is called **only once**.
- Before any Servlet is **reloaded**, the destroy() method is called. The same as when any Servlet is unloaded.

Servlets – Life cycle

- The request processing results from the service() method execution, in particular, there is the option to over-write the methods: doGET(), doPOST(), etc., to distinguish which code to execute depending on the HTTP request command.
- Any service() method invocation is executed in an **independent thread**, but the **same Servlet** *object* is used for any client request, so a multi-thread programming style is necessary.

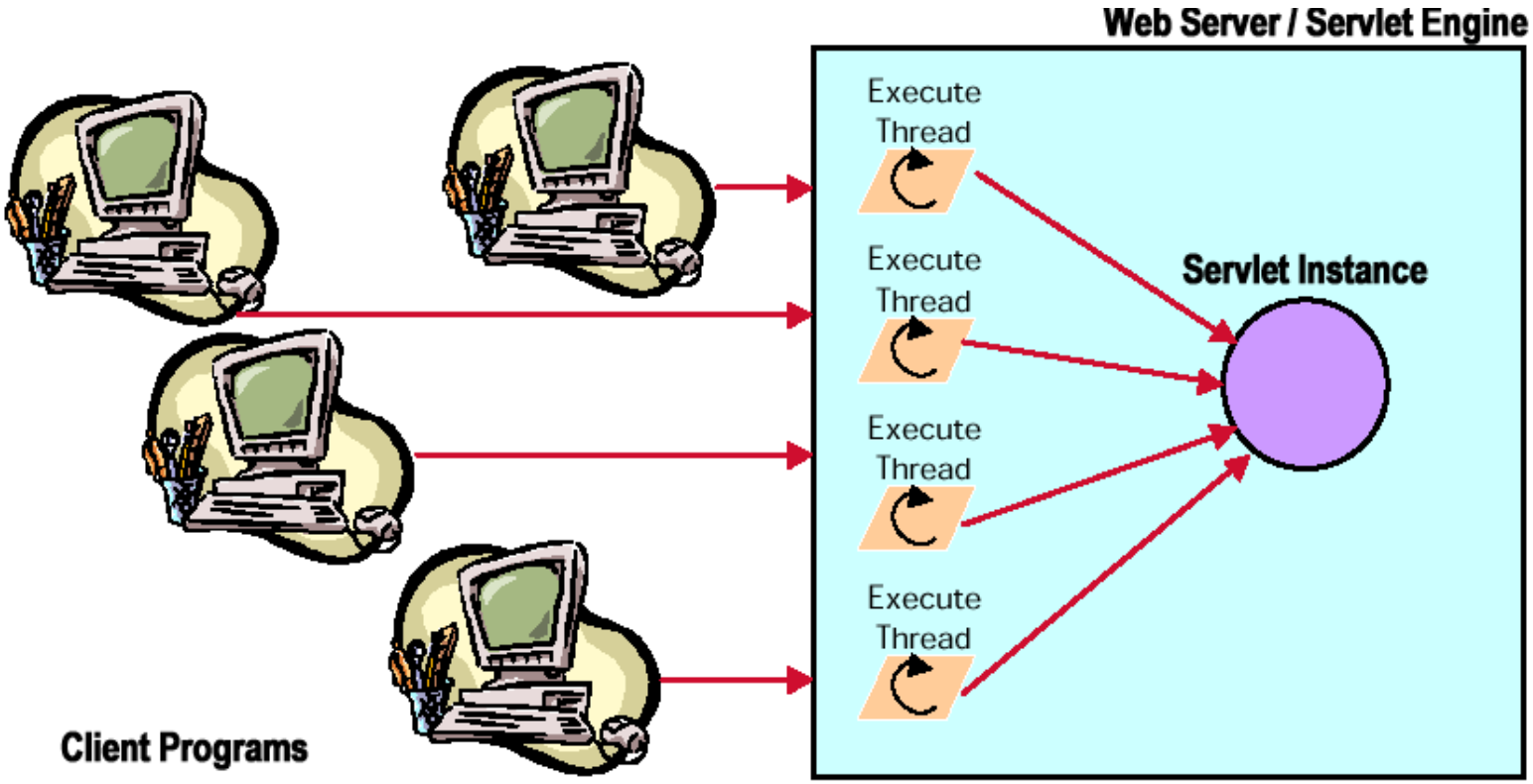
Servlets – Life cycle

- When calling the method destroy() it must be **synchronized** with the method service() due to any potential pending client request to be served.

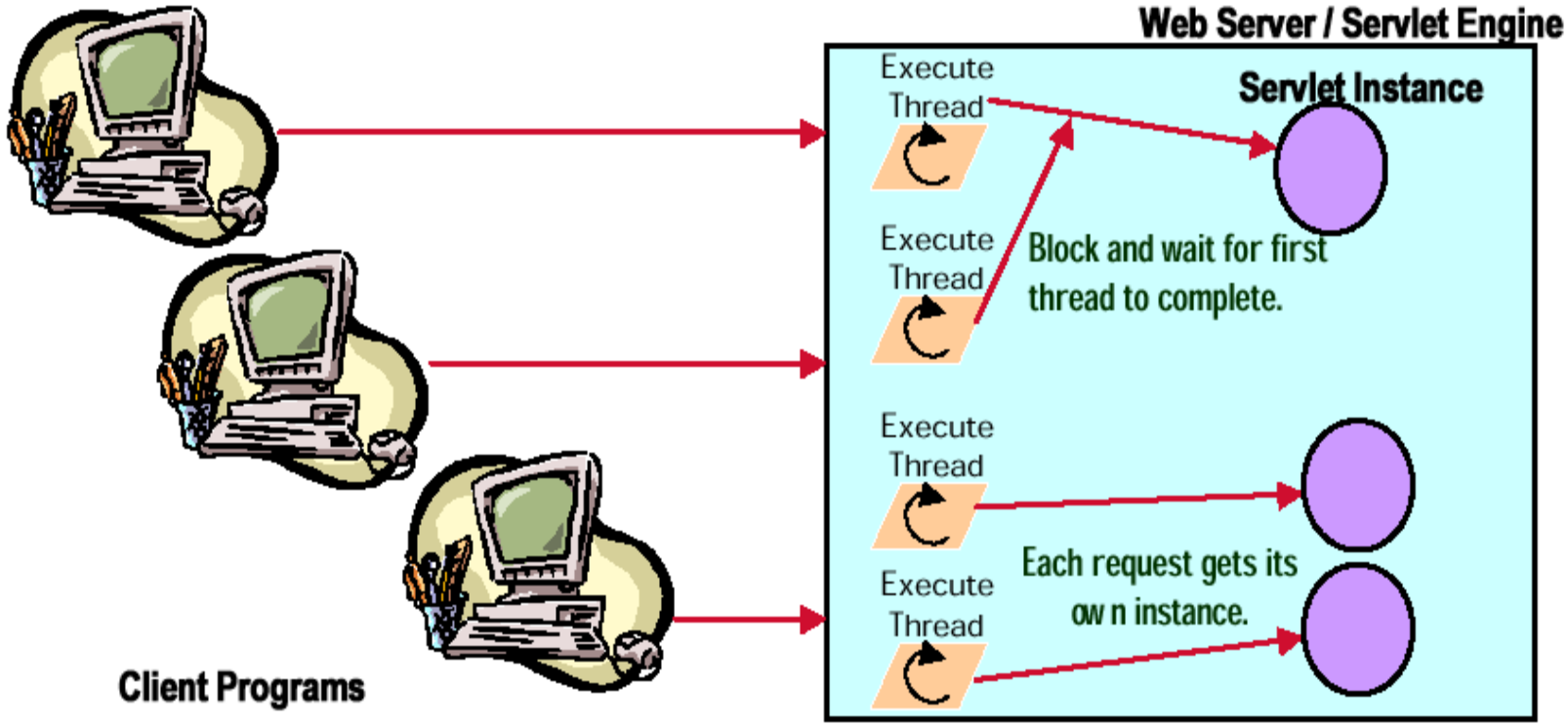
Servlets – Threads (i)

- There are two Servlet execution styles:
  - Servlet thread-safe:
    - A single instance serves concurrent requests
    - Programming thread-safe Java code is necessary.
  - Servlet SingleThreadModel:
    - A single instance serves a single request
    - Maybe successive requests are queued
    - Slow procedure or memory consuming
    - to avoid, or maybe not?

Servlets – Threads (ii)



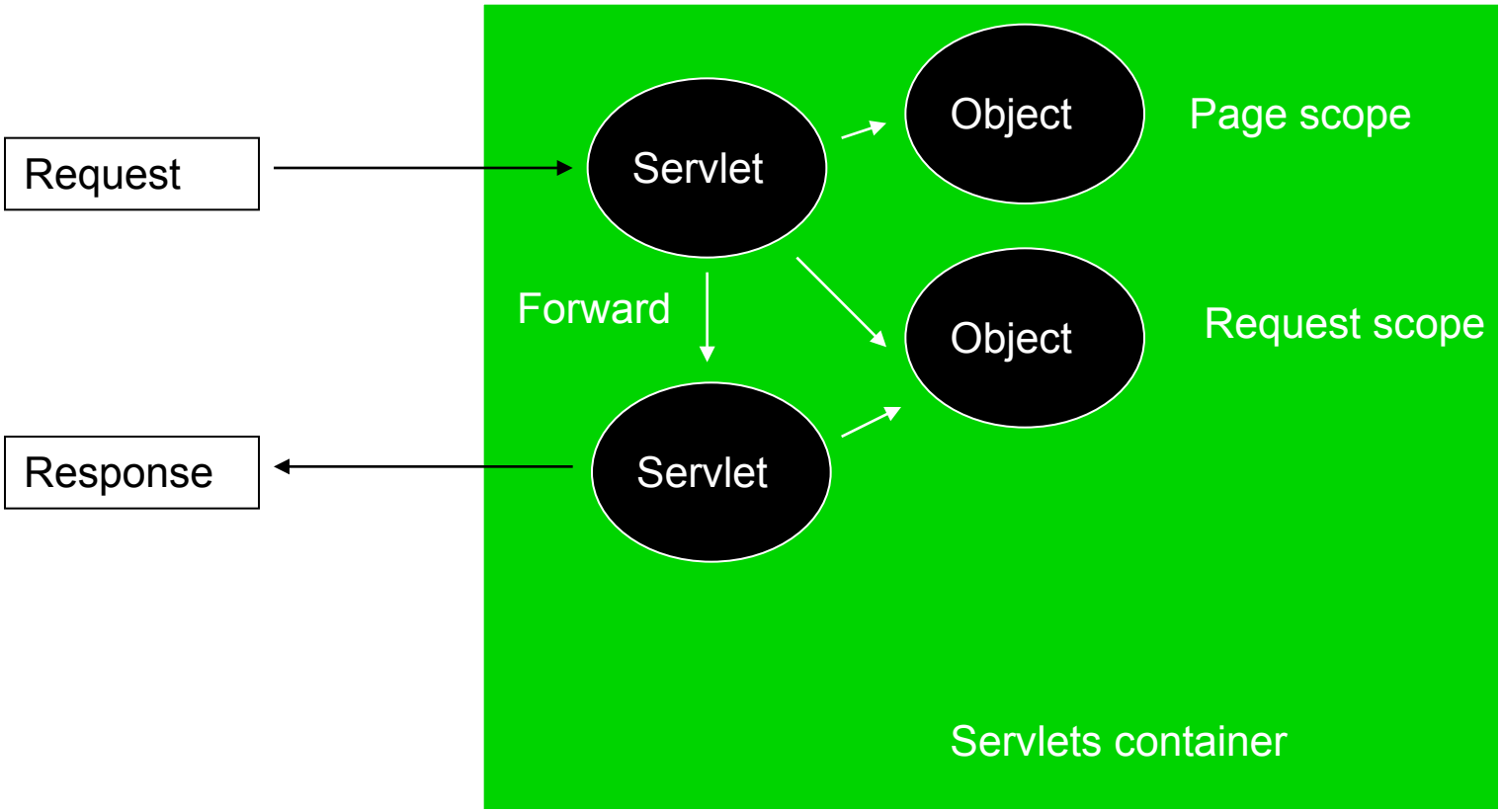
Servlets – Threads (iii)



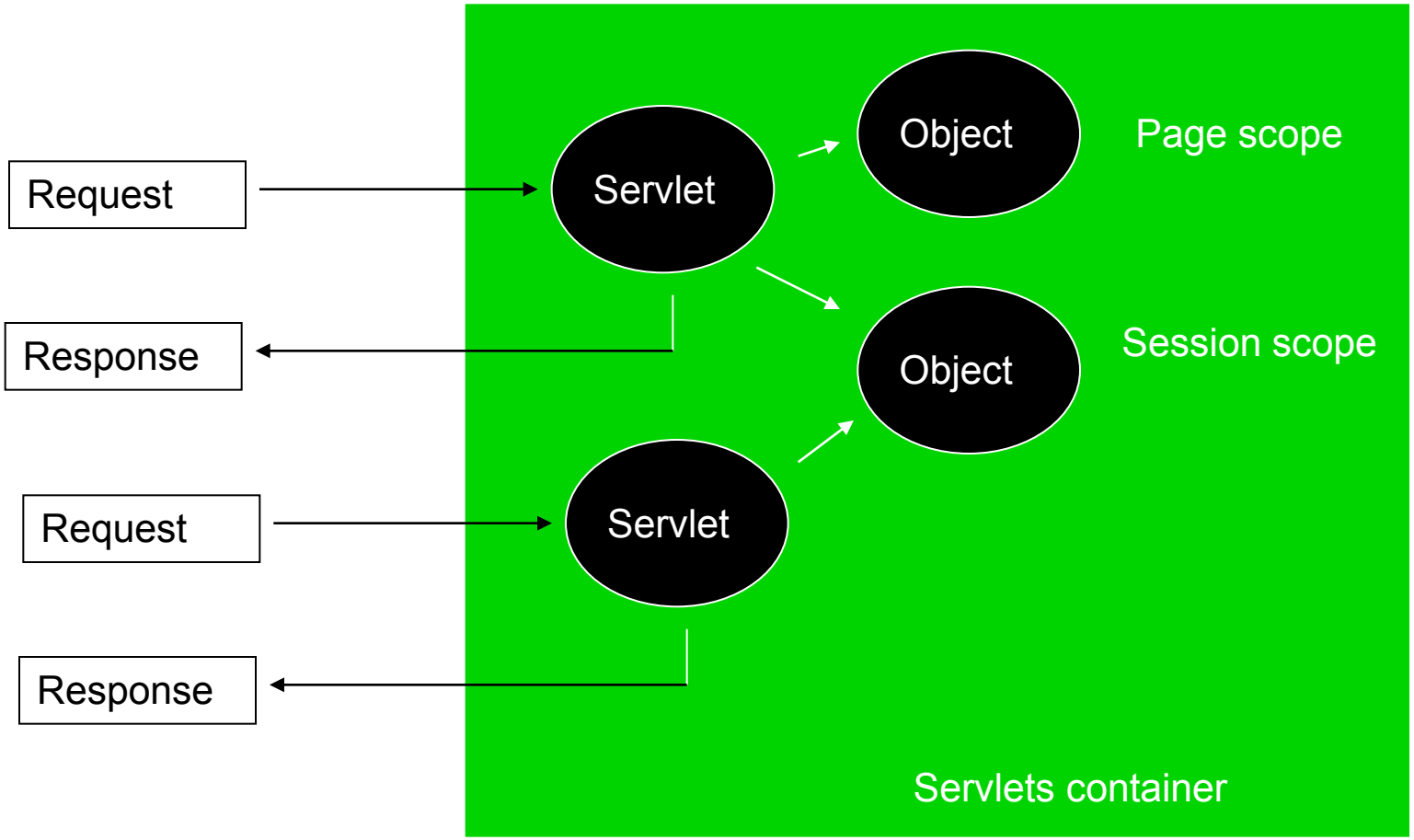
Scope of objects in webApps

- Within the Servlet method (*page* scope)
- Within the request processing (*request* scope)
- Within the session (*session* scope)
- Within the ServletContext (*application* scope)

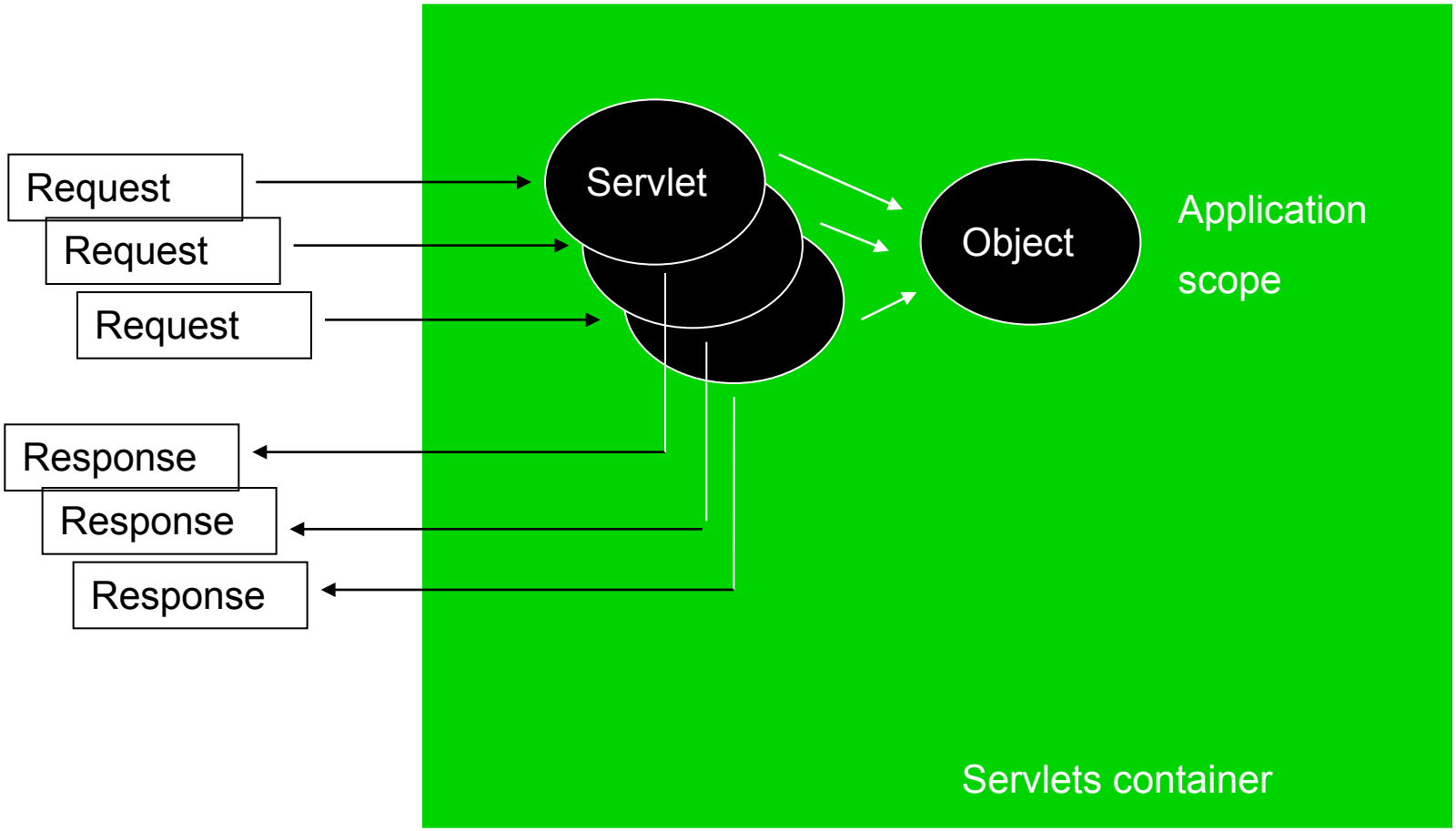
Scope of objects



Scope of objects



Scope of objects



Service architecture

- To process an HTTP client request we must program a new class as an extension of the HttpServlet class.
- When calling a method from HttpServlet class, two objects from interfaces: HttpServletRequest and HttpServletResponse are attached, as parameters of the method call.

Programming Servlets

- Methods to over-write from HttpServlet class:
  - doGet: GET or HEAD
  - doPost: POST
  - doPut: PUT
  - doDelete: DELETE
  - doOptions: OPTIONS
  - doTrace: TRACE

Retrieving data from Request

- Accessing request parameters (query string) from HttpServletRequest class:

```
public String getParameter(String name);
public Enumeration getParameterNames();
public String[] getParameterValues(String name);
```

- Accessing request headers from HttpServletRequest class:

```
public String getHeader(String name);
public Enumeration getHeaders(String name);
public Enumeration getHeaderNames();
```

Programming Servlets

- To have a unique thread associated with a Servlet instance:

```
public class SurveyServlet extends HttpServlet
                           implements SingleThreadModel {

    /* typical servlet code, with no threading concerns
     * in the service method.  No extra code for the
     * SingleThreadModel interface.  */
    ...

}
```

HttpServlet class

Methods inherited from class javax.servlet.GenericServlet:

```
destroy, getInitParameter, getInitParameterNames,
getServletConfig, getServletContext, getServletInfo,
getServletName, init, log
```

Methods inherited from class java.lang.Object:

```
clone, equals, finalize, getClass, hashCode, notify,
notifyAll, toString, wait
```

Dealing with different object scopes

- **Page scope**: no extra tool needed, just create the object within the corresponding Servlet method.

- **Request scope**: from the HttpServletRequest instance we can set / get any object with request scope:

```
public Object getAttribute(String name);
public Enumeration getAttributeNames();
public void setAttribute(String name, Object object);
```

Session scope

- From the HttpServletRequest instance we can recover the HttpSession thru methods:

```
public HttpSession getSession();
public HttpSession getSession(boolean create);
```

- With the HttpSession instance we can set / get any object of **session scope**:

```
public Object getAttribute(String name);
public String[] getAttributeNames();
public void setAttribute(String name, Object value);
public void removeAttribute(String name);
```

Application scope

- With the ServletContext reference we can set / get any object of **application scope**:

```
public Object getAttribute(String name);
public String[] getAttributeNames();
public void setAttribute(String name, Object value);
```

- To obtain the ServletContext reference we must call the getServletContext() method from within any Servlet.

Over-writing the init() method

```
public void init() throws ServletException {
  Connection conn = null;
  String password = getInitParameter(“usrName”);
  String databaseUrl = getInitParameter("databaseUrl");
  conn = DriverManager.getConnection(databaseUrl, “usrName”,
         password);
  getServletContext().setAttribute( ”database_con", conn);
}
```

- The GenericServlet class provides two methods to access predefined strings of characters contained within the web application deployment descriptor file:

```
public String getInitParameter(String name);
public Enumeration getInitParameterNames();
```

Over-writing the destroy() method

```
public void destroy() {
   Connection conn = (Connection)
   getServletContext().getAttribute( ”database_con");
   conn.close();
   conn = null;
}
```

- Take into account any synchronization issue between destroy() and service() methods.

Forwarding the request processing

```
public class Example extends HttpServlet
{  public void service (HttpServletRequest request,
   HttpServletResponse response) throws
   ServletException, IOException {
     Code the_code = new Code();
     request.setAttribute("code",the_code);
     RequestDispatcher rd =
     getServletContext().getRequestDispatcher("/next.jsp");
     rd.forward(request,response);
   }
}
```

Exercises

- Complete the Four_Scopes_exercise managing 4 objects each one with one of the 4 defined scopes.

Java Server Page – definition

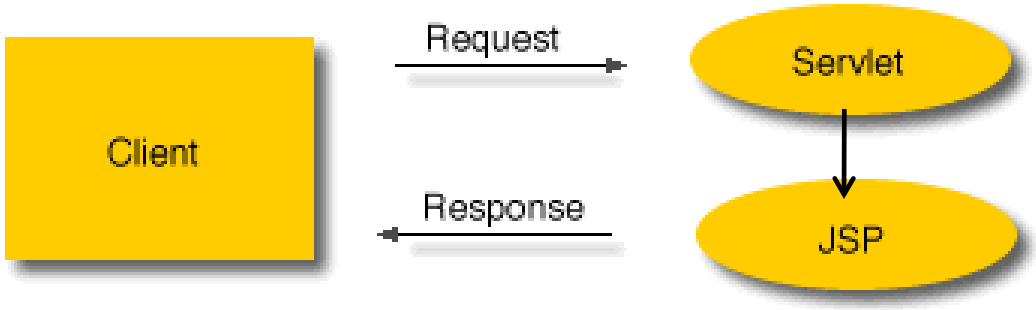
- a JSP is an HTML file containing Java code to dynamically generate the reply to the client
- the JSP elements are distinguished by tags
- a JSP becomes a Servlet on run time, consequently its functionality is equivalent to a Servlet (request/reply, JSP container and dynamic loading on request)
- a JSP engine is needed to work with JSPs

JSP – Modelling



- On requesting a JSP file the application server compiles the JSP file to generate a Servlet which is later on executed.
- The JSP page can obtain the necessary data from a data base directly or using other Java components referenced by the same JSP page.

JSP – Modelling



- In this case the request is sent to a Servlet to process the request and generate the reply content, later the Servlet forwards the request including the reply content to a JSP page to compose the final HTML reply.
- This way the application is easier to maintain on splitting the content generation from the presentation.

JSP syntax

– non-XML notation:

```
<%@ directive %>      <%@ page language="java" %>
<%! declarations %>   <%! int i=0, j=5; %>
<%= expression %>     <%= i+7 %>
<% code fragment %>   <% if(i < j-4 { %>
<%-- comment --%>     <%-- this is the comment --%>
```

– XML notation:

```
directive      <jsp:directive.page … />
declarations   <jsp:declaration> … </jsp:declaration>
expression     <jsp:expression> … </jsp:expression>
code fragment  <jsp:scriptlet> … </jsp:scriptlet>
comment        <%-- this is the comment --%>
actions        <jsp:useBean>, etc.
```

JSP directives

- include directive: `<%@ include file= … %>`
- page directive:
  - Language (only Java)
  - extends `another class (the resultant servlet)`
  - contentType
  - import `<%@ page import="java.util.*" %>`
  - session `true, false (supports session objects)`

Action Elements

- Bean related actions:
  - useBean (java object), getProperty, setProperty (on an object field):

```
<jsp:useBean id="my_bean" scope="session"
  class="directory.MainBean" />
<jsp:setProperty name="my_bean"
  property="atribute_1" value="the_value" />
```

- *include* and *forward* actions

```
<jsp:include page="test.jsp" flush="true" />
```

- *plugin* action to download on client